

Business Financial Performance Analysis System

Using Python

Abstract

The Business Financial Performance Analysis System is a Python app for analysing financial performance of the business from monthly sales and expense reports from the user. The system calculates total sales, total expenses, monthly profit, average sales, profit percentage and identifies which month was the best performing. This is done by generating a financial report that can be used to help in determining the overall profitability of the business.

Problem Statement

Regular evaluation of a company's financial success is necessary to assist businesses in making proper decisions regarding their future. Financially evaluating performance requires a significant amount of time and is subject to human error. This project automates manual calculations of profit, expense, and business performance assessment through the submission of financial data, and produces an immediate and detailed report on business performance as soon as a user enters their financial information.

Objectives

- To collect monthly sales and expense data from users.
- To calculate monthly profit or loss.
- To determine total sales and total expenses.
- To calculate overall business profit.
- To identify the month with the highest sales.
- To calculate average monthly sales.
- To determine profit percentage.
- To generate a business financial report.

Technologies Used

Programming Language

- Python 3.x

Concepts Used

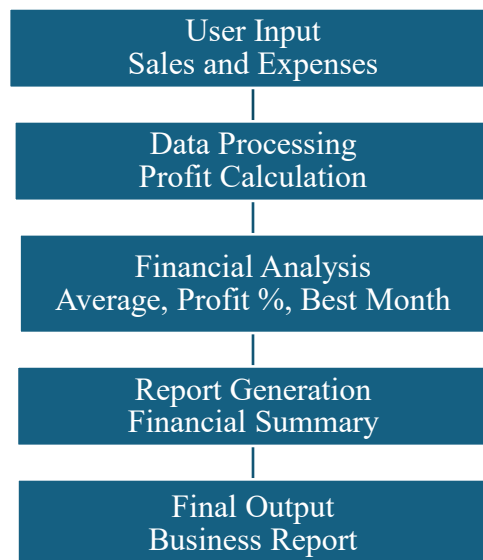
- Variables

- Input and Output Operations
- Data Types
- Lists
- Tuples
- Functions
- Loops
- Conditional Statements
- Operators
- Inbuilt Functions

Development Environment

- Visual Studio Code (VS Code)
- Python Interpreter

System Flow Diagram



System Design

Input

User enters:

- Monthly Sales Amount
- Monthly Expense Amount

for six months.

Processing

- 1) Store sales and expense data.
- 2) Calculate monthly profits.
- 3) Calculate total sales.
- 4) Calculate total expenses.
- 5) Calculate total profit.
- 6) Calculate average sales.
- 7) Identify best sales month.
- 8) Calculate profit percentage.

Output

- Total Sales
- Total Expenses
- Total Profit
- Business Status
- Average Sales
- Best Sales Month
- Monthly Profit Report
- Profit Percentage

Algorithm

Step 1: Define months using a tuple.

Step 2: Create empty lists for sales and expenses.

Step 3: Accept sales and expense values from the user.

Step 4: Create functions to:

- Calculate total amount
- Calculate average amount
- Calculate monthly profit

Step 5: Calculate profits for each month.

Step 6: Compute:

- Total Sales
- Total Expenses
- Total Profit
- Average Sales

- Profit Percentage

Step 7: Find the best-performing month.

Step 8: Display the final financial report.

Financial Formulas Used

Profit = Sales - Expenses

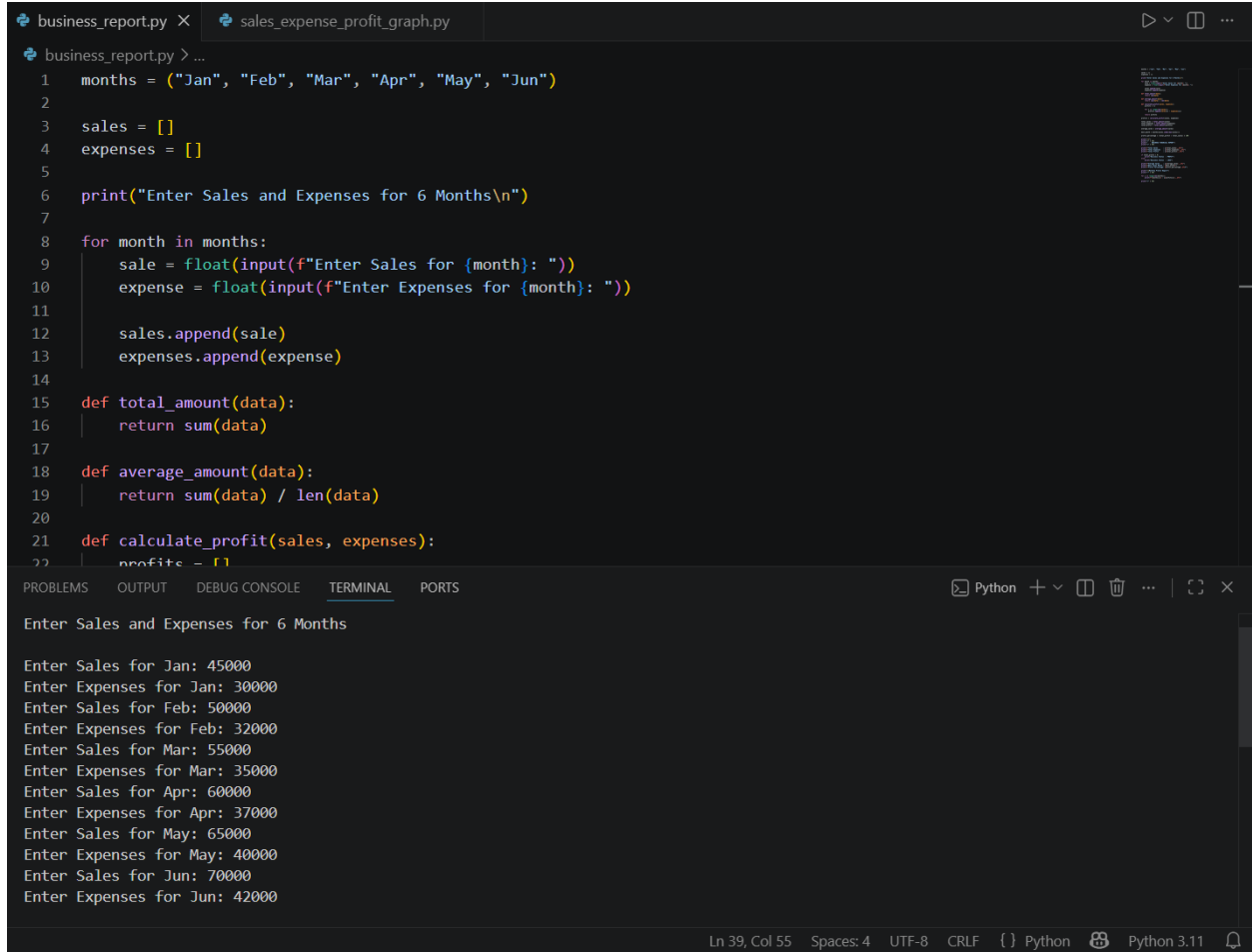
Average Sales = Total Sales / Number of Months

Profit Percentage = (Total Profit / Total Sales) $\times$ 100

Features

- User-friendly input system
- Monthly sales and expense tracking
- Profit calculation
- Business performance evaluation
- Profit percentage analysis
- Best month identification
- Structured financial reporting

Sample Input



The image shows a code editor with two tabs: `business_report.py` and `sales_expense_profit_graph.py`. The `business_report.py` tab is active, displaying a Python script. The script defines a list of months, initializes `sales` and `expenses` as empty lists, and prompts the user to enter sales and expenses for 6 months. It then defines functions to calculate the total amount, average amount, and profit. The terminal output shows the user's input for each month.

```
business_report.py > ...
1 months = ("Jan", "Feb", "Mar", "Apr", "May", "Jun")
2
3 sales = []
4 expenses = []
5
6 print("Enter Sales and Expenses for 6 Months\n")
7
8 for month in months:
9     sale = float(input(f"Enter Sales for {month}: "))
10    expense = float(input(f"Enter Expenses for {month}: "))
11
12    sales.append(sale)
13    expenses.append(expense)
14
15 def total_amount(data):
16     return sum(data)
17
18 def average_amount(data):
19     return sum(data) / len(data)
20
21 def calculate_profit(sales, expenses):
22     profits = []
```

Enter Sales and Expenses for 6 Months

Enter Sales for Jan: 45000
Enter Expenses for Jan: 30000
Enter Sales for Feb: 50000
Enter Expenses for Feb: 32000
Enter Sales for Mar: 55000
Enter Expenses for Mar: 35000
Enter Sales for Apr: 60000
Enter Expenses for Apr: 37000
Enter Sales for May: 65000
Enter Expenses for May: 40000
Enter Sales for Jun: 70000
Enter Expenses for Jun: 42000

Ln 39, Col 55 Spaces: 4 UTF-8 CRLF () Python Python 3.11

Sample Output

```
=====
BUSINESS FINANCIAL REPORT
=====
Total Sales      : ₹345,000.00
Total Expenses   : ₹216,000.00
Total Profit     : ₹129,000.00
Business Status  : PROFIT
Average Sales    : ₹57,500.00
Best Sales Month : Jun
Profit Percentage: 37.39%

Monthly Profit Report
-----
Jan : ₹15,000.00
Feb : ₹18,000.00
Mar : ₹20,000.00
Apr : ₹23,000.00
May : ₹25,000.00
Jun : ₹28,000.00
=====
```

Business Analysis

The report indicates that:

- The business generated a positive profit.
- Sales increased steadily over the six-month period.
- Expenses remained lower than sales.
- Profitability improved month by month.
- June was the highest-performing month.

Applications

- Business Performance Analysis
- Financial Reporting
- Cost and Management Accounting
- Budget Monitoring
- Educational Demonstrations
- Business Decision Support

Future Enhancements

- Store data in CSV files
- Read data from Excel spreadsheets
- Generate graphical reports using Matplotlib
- Use Pandas for advanced analysis
- Create a graphical user interface (GUI)
- Develop a web-based dashboard
- Add yearly and quarterly analysis

Learning Outcomes

Through this project, I learned:

- Python programming fundamentals
- Functions and modular programming
- Data structures such as lists and tuples
- Looping and conditional statements
- Business data processing
- Financial analysis concepts
- Report generation techniques

Limitations

- Data is entered manually by the user.
- Analysis is limited to six months.
- No database storage.
- No graphical visualization.
- No forecasting capability.

Conclusion

The Business Financial Performance Analysis System has effectively showcased how Python can be used in commerce and business analytics. This project automates the calculation of finances while also providing insights about sales, expenses and profits.

This system is a basis for more advanced financial analysis tools based on the practical application of Python for helping to do accounting and make management decisions.